CLAUDE CERTIFIED ARCHITECT — PROFESSIONAL

Objective 24: Optimise Token Usage, Latency and Cost-Performance

Exam objective: Optimize token usage, latency, and cost-performance using prompt caching, context reduction, model routing, batch processing, parallel execution, retrieval optimization, output token limits, model and effort selection, and cost per successful task.

Q1 **A customer-support SaaS platform sends the same lengthy system prompt and tool definitions on every single API call for a high-volume chatbot. Which optimization would most directly reduce cost without changing the model or degrading quality?**

A Reduce the number of tools available

B **Implement prompt caching for the static system prompt and tool definitions**

C Switch to a smaller model for all requests

D Increase the output token limit

CORRECT ANSWER **B**

WHY Prompt caching allows the static, repeated portion of the prompt (system instructions, tool definitions) to be reused across calls at reduced cost, directly cutting spend without touching the model or truncating capability. Removing tools or switching models could degrade functionality or quality, and raising output limits increases cost rather than reducing it.

Q2 **A financial-services firm's research assistant retrieves 15 documents per query from its knowledge base, but analysis shows only 2–3 are typically relevant to answering the question, inflating token usage and latency. What is the best optimization?**

A Retrieve even more documents to be safe

B Improve retrieval precision (better ranking/filtering) to return fewer, more relevant documents

C Increase the output token limit to compensate

D Switch to batch processing for all queries

CORRECT ANSWER B

WHY Improving retrieval precision so that only genuinely relevant documents are passed into context directly reduces token consumption and latency while maintaining or improving answer quality. Retrieving more documents worsens the problem, output limits and batch processing don't address the root retrieval-precision issue.

Q3 **A telecom company processes nightly batches of 50,000 call-transcript summaries that do not need real-time responses. Which optimization is most appropriate to reduce cost for this specific workload?**

A Real-time streaming API calls for each transcript

B Batch processing (asynchronous bulk API) to take advantage of batch pricing and throughput

C Increasing the model's effort/reasoning level for every transcript

D Prompt caching only, with no other changes

CORRECT ANSWER B

WHY Since the workload is non-real-time and high-volume, asynchronous batch processing is specifically designed for this use case, typically offering significant cost savings over real-time calls. Real-time streaming is unnecessary overhead here, and increasing effort level would raise cost without addressing throughput/pricing efficiency.

Q4 **A retail company's support agent handles simple FAQ questions and complex multi-step troubleshooting with the same large, high-capability model for every request, resulting in high average cost per request. What optimization would most effectively reduce cost while preserving quality on harder queries?**

A Reduce the context window for all requests uniformly

B **Implement model routing: use a smaller/cheaper model for simple queries and the larger model for complex ones**

C Cap the output token limit extremely low for all requests

D Switch every request to batch processing regardless of latency needs

CORRECT ANSWER B

WHY Model routing matches query complexity to the appropriate model tier, cutting cost on the (typically numerous) simple queries while preserving the larger model's capability for genuinely complex ones. Uniformly reducing context or output limits risks degrading quality across the board, and forcing all traffic into batch processing ignores real-time latency requirements for support interactions.

Q5 **A logistics company's route-optimization agent needs to query three independent downstream services (traffic data, weather data, fuel pricing) to generate a recommendation. Currently these are called sequentially, each taking ~800ms, for a total of ~2.4 seconds added latency. What is the best optimization?**

A Switch to a smaller model to compensate for the delay

B **Execute the three independent tool calls in parallel since they don't depend on each other**

C Remove one of the three data sources permanently to save time

D Increase the context window size

CORRECT ANSWER **B**

WHY Since the three calls are independent of one another, executing them in parallel rather than sequentially reduces total latency to roughly the slowest single call (~800ms) instead of the sum, without sacrificing any data source. Removing a data source sacrifices quality, and the other options don't address the sequential-execution bottleneck.

Q6 **A manufacturing company's architect notices that output responses from its equipment-diagnostics assistant are often far longer than necessary, containing redundant explanations that increase both cost and user wait time. What is a straightforward optimization to address this?**

A Increase the model's temperature setting

B **Set explicit, appropriately tuned output token limits and instruct the model toward concise responses**

C Switch to a larger, more capable model

D Add more few-shot examples to the prompt, regardless of their relevance

CORRECT ANSWER B

WHY Bounding output length via token limits combined with explicit conciseness instructions directly targets verbose, redundant output, reducing both cost and latency. Increasing temperature affects randomness not length, switching to a larger model would likely increase cost further, and adding irrelevant examples bloats the prompt rather than trimming output.

Q7 **A healthcare provider's clinical documentation assistant uses maximum reasoning effort on every single query, including simple formatting tasks like converting bullet points into a table. What cost-performance issue does this illustrate, and what is the fix?**

A. This is optimal since higher effort always produces better results regardless of task

B. Using high effort uniformly wastes cost/latency on tasks that don't require deep reasoning; the fix is matching effort level to task complexity

C. The fix is to disable reasoning entirely for every task type

D. The fix is to increase the context window instead of adjusting effort

CORRECT ANSWER B

WHY Reasoning effort should be matched to task complexity — simple formatting tasks gain little from maximum effort while incurring unnecessary latency and cost, so tuning effort level per task type (or routing) is the correct optimization. Applying effort uniformly high, disabling it entirely, or adjusting an unrelated parameter (context window) misses the actual lever.

Q8 **An insurance company tracks "cost per successful claim-triage decision" rather than raw "cost per API call" as its primary optimization KPI. A new, cheaper model configuration reduces cost per call by 20% but increases the failure/retry rate by 35%. What does this KPI reveal that cost-per-call alone would miss?**

A The cheaper configuration is unambiguously better because cost per call decreased

B **The cheaper configuration may actually increase overall cost per successful task once retries and failures are accounted for**

C Cost per successful task is identical to cost per call and provides no additional insight

D Failure rate is irrelevant to cost optimization

CORRECT ANSWER **B**

WHY Because failures often trigger retries or escalations (consuming additional tokens/calls), a configuration that's cheaper per call but fails more often can end up more expensive per successful outcome — exactly what cost-per-call alone would miss. The other options incorrectly treat the two metrics as equivalent or dismiss the failure-rate trade-off.

Q9 **A SaaS company's documentation assistant includes the entire product manual (200+ pages) in every prompt's context, even though most questions only relate to a small section. Which combination of optimizations addresses both cost and latency here?**

A Keep full-manual inclusion but switch to a cheaper model

B Context reduction via targeted retrieval (RAG) to include only relevant manual sections, reducing tokens processed per request

C Increase the output token limit to compensate for the large input

D Switch to batch processing to hide the latency impact

CORRECT ANSWER B

WHY Retrieval-based context reduction — fetching only the relevant sections instead of the entire manual — directly reduces both input token count (cost) and processing time (latency) while maintaining answer quality. Switching models alone doesn't fix the bloated context, and output limits or batch processing don't address the oversized input.

Q10 A government agency's architect wants to reduce latency for a citizen-facing chatbot during peak hours without reducing answer quality. Current architecture calls three sequential prompts (classify intent → retrieve → generate) per request. What change offers the best latency improvement while preserving quality?

A. **Combine independent steps where possible and parallelize any steps that don't depend on each other's output**

B. Remove the retrieval step entirely to save time

C. Increase model effort level to compensate for perceived slowness

D. Add a fourth sequential step for additional validation

CORRECT ANSWER A

WHY Reducing unnecessary sequential dependencies — combining steps or running independent ones in parallel — is the direct way to cut latency without sacrificing the functional steps needed for quality. Removing retrieval would harm answer grounding, increasing effort would raise latency further, and adding another sequential step would increase it too.

Q11 **A retail company notices that its prompt caching strategy, implemented for a product-catalog assistant, isn't reducing costs as expected. Investigation reveals the system prompt includes a dynamically generated timestamp at the very beginning of the prompt on every call. What is the issue and the fix?**

A Prompt caching cannot work with any system prompt; the fix is to abandon caching entirely

B **The dynamic timestamp invalidates the cache on every call since the cached prefix must be identical; move variable content to the end or remove it from the cached portion**

C The fix is to increase the model's temperature

D The fix is to switch to a smaller model instead

CORRECT ANSWER **B**

WHY Prompt caching relies on an identical, stable prefix across calls; placing dynamic content (like a timestamp) at the start invalidates the cache every time, so the fix is restructuring the prompt to keep the cacheable (static) portion separate from variable content. Abandoning caching, adjusting temperature, or switching models don't address the actual structural cause.

Q12 **A law firm's contract-analysis tool processes individual contracts one at a time via real-time API calls throughout the business day, even though 80% of volume could tolerate a few hours of delay for routine due-diligence reviews. What cost optimization fits this scenario best?**

A Switch all requests, including urgent ones, to batch processing

B **Segment workload: route the delay-tolerant 80% to batch processing for cost savings while keeping urgent requests on real-time calls**

C Increase the context window for all requests

D Apply maximum reasoning effort to all requests to improve accuracy

CORRECT ANSWER B

WHY Segmenting traffic by latency tolerance allows the bulk of delay-tolerant work to benefit from cheaper batch processing while preserving real-time responsiveness for genuinely urgent requests — a balanced, workload-aware optimization. Forcing all traffic into batch mode would break urgent-request SLAs, and context/effort changes don't address the batching opportunity at all.

Q13 **A manufacturing company's architect is evaluating whether to adopt an extended-thinking (higher effort) model configuration for its root-cause-analysis assistant. Evaluation shows a 6% accuracy improvement but a 3x increase in latency and cost. What is the most balanced architectural recommendation?**

A Always adopt the higher-effort configuration since any accuracy gain justifies any cost

B **Evaluate whether the specific use case's stakes and volume justify the cost/latency trade-off, potentially routing only high-stakes cases to the higher-effort configuration**

C Reject the higher-effort configuration outright since cost increased

D Ignore cost and latency entirely since they are not part of architectural decision-making

CORRECT ANSWER B

WHY A nuanced, trade-off-aware decision — such as selectively routing only genuinely high-stakes cases to the costlier high-effort configuration — captures the accuracy benefit where it matters most while controlling overall cost and latency impact. Blanket adoption or blanket rejection both ignore the situational trade-off, and dismissing cost/latency as irrelevant contradicts sound architectural practice.

Q14 **A telecom provider's architect wants to reduce token consumption in a multi-turn support conversation without losing necessary context for the agent to function correctly. Which approach best achieves this?**

A Truncate the conversation history randomly at a fixed token count

B Summarize or compress older conversation turns into a condensed context while retaining recent turns and key facts in full

C Remove all prior conversation history on every turn

D Increase the context window size instead of reducing tokens

CORRECT ANSWER **B**

WHY Intelligent context compression — summarizing older turns while preserving recent exchanges and critical facts — reduces token usage while retaining the information the agent actually needs, unlike random truncation or wholesale history removal which risk losing important context. Increasing the context window increases cost rather than reducing it.

Q15 **An insurance company's architect wants a single north-star metric to evaluate whether a proposed cost-optimization change (e.g., switching models, caching, or routing) is actually beneficial overall, rather than optimizing cost in isolation. Which metric best serves this purpose?**

A Raw token consumption per request

B Cost per successful task, which accounts for both spend and the rate of successful outcomes

C Average latency per request only

D Number of API calls per day

CORRECT ANSWER **B**

WHY Cost per successful task combines economic efficiency with outcome quality, preventing optimizations that merely shift cost without preserving (or at the expense of) task success. The other metrics each capture only one dimension (raw usage, speed, or call volume) without connecting it to whether the task was actually accomplished.

Q16 **A legal-tech company's architect proposes a "cheap-first, escalate-on-failure" strategy for its document-classification task: run every request on a cheaper, faster model first, and only re-run it on the larger, more expensive model when a reliable failure signal detects the cheap model likely got it wrong. What is the essential prerequisite for this strategy to actually reduce cost per successful task rather than just adding latency?**

A The strategy works automatically regardless of whether failures can be detected

B **A trustworthy, low-false-negative failure-detection signal to decide when escalation is needed; without it, failed outputs go uncaught and savings don't translate into reliable successful outcomes**

C The cheap model must always be slower than the expensive model

D The strategy only works if both models are identical in capability

CORRECT ANSWER **B**

WHY The cost savings of a cheap-first-with-retry approach depend entirely on being able to detect, with high reliability, when the cheap model's output is likely wrong so the small fraction of failures can be escalated; without such a detector, undetected failures silently reduce quality instead of being caught and retried. The strategy's benefit is not automatic, is unrelated to relative model speed, and does not require the two models to have identical capability — in fact it depends on their capability being different.

Q17 **An operations team tries to move its nightly batch-API document-summarization job to also accept a "fast streaming" response mode and a stateful conversation-thread parameter that work fine in its real-time chat product. Both are rejected by the batch API with validation errors. What is the underlying architectural reason?**

A The batch API is simply broken and should be reported as a bug

B **Streaming, fast-mode, and stateful/session parameters are designed for synchronous, real-time interaction; asynchronous batch processing has no live connection or session to stream to or maintain state within, so these parameters don't apply**

C Batch processing and streaming are two names for the same underlying mechanism

D The fix is to always use real-time calls instead of batch for every workload

CORRECT ANSWER B

WHY Streaming, low-latency "fast" modes, and stateful session parameters all assume an open, synchronous connection to progressively deliver or maintain context for a single live caller — a model of interaction that asynchronous, queued batch processing does not provide, which is why those parameters are rejected rather than silently ignored. This is expected, intentional behavior rather than a bug, the two mechanisms are not interchangeable, and abandoning batch processing would forfeit its cost/throughput advantages for workloads that don't need real-time responses. --